PROJECT CHECK-IN 2

Normalization and Architectural Evolution

PRICEDROP DIARY | Price History Tracker with Smart Buy Signal

Dev Raj Dhungana • WSU ID: N348Z294

1. Project continuity and schema evolution

My Check-in 1 described an offline-first Android app (Kotlin, Jetpack Compose, SQLite through Room) for manually logging product prices at different stores, viewing history and getting a smart buy-now notification. My original database already had three tables: Product, Store and PriceObservation, rather than only two. I keep those tables and add Category as a small lookup table to make the category filter more reliable.

Check-in 1 table	What remains / improvement
Product	Retain product_id, name and optional target price; replace the category text with category_id referencing Category.
Store	Retain store_id, unique store_name and is_online; no new fields needed.
PriceObservation	Retain each dated price, optional note and its product/store references; preserve the one-price-per-product/store/day rule.
New: Category	Store category name once and link it to multiple products. This also supports a consistent category filter.

Scope unchanged: manual entries, offline history, minimum/average price, and the existing two-part buy rule. No scraping, cloud sync, machine learning, or accounts are added. The price statistics and buy-now signal remain calculated, not stored.

2. One consistent Check-in 1 example

All six price records and the three products/stores below are carried over unchanged from Check-in 1. Category identifiers are new demonstration keys; the category labels themselves are the originals.

Product ID / product	Category (new ID)	Target
1 • Noise-Cancelling Headphones	1 • Electronics	$250.00
2 • Running Shoes	2 • Apparel	$90.00
3 • Espresso Grinder	3 • Kitchen	NULL
Store ID	Store name	Online?
1	Amazon	1
2	Best Buy	0
3	Target	0

Obs ID	Product ID	Store ID	Date	Price
1	1	1	2026-09-01	299.99
2	1	2	2026-09-15	279.99
3	1	1	2026-10-01	249.99
4	2	3	2026-09-10	110.00
5	2	3	2026-10-02	89.99
6	2	1	2026-10-02	95.00

3. Step-by-step normalization

3.1 Unnormalized working sheet (UNF)

The actual Check-in 1 SQL tables were already atomic. To demonstrate why normalization matters, I first imagined the app's observations as a single working sheet that repeats product name, category, store name, online flag, target price, date and price together. An even less structured version would keep several price/date pairs in a single product cell; that would not be 1NF. This is an illustrative pre-normalization sheet, not a claim that Check-in 1 actually stored lists.

3.2 First Normal Form (1NF)

I give each observation its own row and one value per field. In the demonstration sheet, (product_id, store_id, observed_date) is a candidate business key because Check-in 1 permits just one observation for the same product and store on a date. obs_id, the original surrogate identifier, is also preserved. The compact view shows all six records; each also carries the product/store descriptions from the tables above.

obs_id	product_id	store_id	date	price	category	store_name
1	1	1	09-01	299.99	Electronics	Amazon
2	1	2	09-15	279.99	Electronics	Best Buy
3	1	1	10-01	249.99	Electronics	Amazon
4	2	3	09-10	110.00	Apparel	Target
5	2	3	10-02	89.99	Apparel	Target
6	2	1	10-02	95.00	Apparel	Amazon

All dates above are in 2026. This table is 1NF: individual cells are atomic. But repeating product and store facts creates redundant data. In terms of the composite business key, product_name, category and target_price depend on product_id alone; store_name and is_online depend on store_id alone. That is the partial-dependency issue shown in the next step.

3.3 Second Normal Form (2NF)

I separate product facts from store facts and keep only observation facts in PriceObservation. This removes partial dependencies relative to the composite business key: a change to Amazon's name or the headphones' target price no longer needs edits in six observation rows. The original three-table design already accomplished this, so the step confirms an existing strength rather than inventing a flaw.

2NF relation	Primary key	Original example rows
Product_2NF	product_id	1 / Headphones / Electronics / 250.00; 2 / Running Shoes / Apparel / 90.00; 3 / Espresso Grinder / Kitchen / NULL
Store	store_id	1 / Amazon / 1; 2 / Best Buy / 0; 3 / Target / 0
PriceObservation	obs_id; also UNIQUE(product_id,store_id,observed_date)	All six observations from Section 2, with product_id, store_id, price, date and optional note

Because obs_id is a single-column surrogate key, partial dependencies on a subset of that key are impossible in the original schema. The composite candidate key is used only to illustrate the 1NF → 2NF decomposition. Both descriptions are compatible.

3.4 Third Normal Form (3NF)

The original Product table stores category as a free-text value. Although this is not necessarily a formal 3NF violation—category text is itself the category fact—it can lead to inconsistent spelling across products. For a meaningful architecture upgrade, I introduce category_id and category_name. In a temporary Product_2NF view containing both category_id and category_name, category_id → category_name and product_id → category_id, so category_name is transitively dependent on product_id. Moving it into Category removes this transitive dependency in the revised design.

Final 3NF table	Primary key	Foreign keys / attributes
Category	category_id	category_name UNIQUE

Product	product_id	category_id → Category; product_name, target_price
Store	store_id	store_name UNIQUE, is_online
PriceObservation	obs_id	product_id → Product; store_id → Store; price, observed_date, note

The observation-to-product and observation-to-store links reconstruct the original price history without changing any prices. Joining Product with Category returns Electronics, Apparel and Kitchen just as before.

4. Functional dependencies

Determinant	Dependent attribute(s)
category_id	category_name
product_id	product_name, category_id, target_price
store_id	store_name, is_online
obs_id	product_id, store_id, price, observed_date, note
(product_id, store_id, observed_date)	obs_id, price, note (under the one-reading/day rule)
store_name	store_id (because store names are UNIQUE)

In the flattened 1NF example, product_id → product_name / category / target_price and store_id → store_name / is_online are partial dependencies on the composite observation business key. In the temporary 2NF category-ID design, product_id → category_id → category_name is transitive; the final Category lookup separates that dependency.

5. Early-design anomalies and resolutions

Problem	What could go wrong	How the updated design helps
Insertion	A new store or product might be impossible to describe independently in one flattened price-history sheet.	Product and Store have their own rows, even with zero observations; Category can be added before any products use it.
Update	Changing the label Electronics in every product row could miss one or create variants such as Electronic.	Category.category_name is updated once; products reference category_id.
Deletion	Removing the last observation for a product in a flattened table might erase its product details.	Product details stay in Product; deleting one observation does not delete the product.
Redundant stored value	A stored low/average/buy flag can become stale when an observation is edited or removed.	MIN, AVG, COUNT and the buy signal are calculated by SQL, as already planned in Check-in 1.

6. Updated entity–relationship diagram

The new Category–Product relationship is one-to-many. Product–PriceObservation and Store–PriceObservation remain one-to-many, as in Check-in 1. Each price observation belongs to one product and one store. Together those observations implement the product/store many-to-many association across dates.

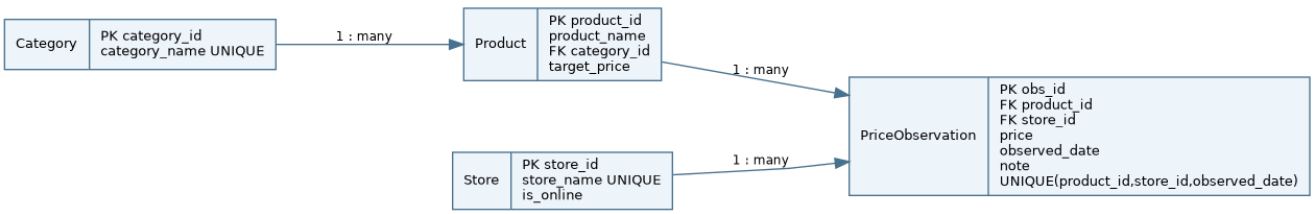


Figure 1. Updated PriceDrop Diary 3NF ER diagram. An editable .drawio file is provided separately.

7. Data integrity and architecture

SQLite foreign keys are enabled using PRAGMA foreign_keys = ON (Room must also enforce them). Primary keys identify each row; the observation table uses FOREIGN KEY references and UNIQUE(product_id, store_id, observed_date), preserving Check-in 1's rule. CHECK(price > 0), CHECK(target_price > 0 when present), and CHECK(is_online IN (0,1)) reject invalid values. Store and category names are unique. Product/category uses ON DELETE SET NULL to allow category removal without deleting a product; deleting a product or store cascades to its observations, as Check-in 1 specified.

The optional target_price is NULL for Espresso Grinder. Product.category_id can be NULL after deleting a category. Observed dates are ISO-formatted local strings (YYYY-MM-DD) in this SQLite example; real app input validation should reject impossible calendar dates. An index on (product_id, store_id, observed_date DESC) supports quick latest-price lookup. No lowest price, average price, count or buy-now flag is persisted.

8. Verification with the same data

Check	Expected outcome
Noise-Cancelling Headphones observations	Amazon $299.99 (Sep 1); Best Buy $279.99 (Sep 15); Amazon $249.99 (Oct 1)
Running Shoes observations	Target $110.00 (Sep 10); Target $89.99 (Oct 2); Amazon $95.00 (Oct 2)
Headphone stats	Lowest $249.99; average $276.66; count 3
Running Shoes stats	Lowest $89.99; average $98.33; count 3
Latest-per-store buy-now results	Headphones at Amazon $249.99; Running Shoes at Target $89.99
Espresso Grinder	No observations and no target; does not appear in observation-based stats or buy-now results

The exact Check-in 1 smart rule is preserved: only the latest observation per product/store is considered. A target hit occurs when the price is at or below the product target. Alternatively a historical deal needs at least 3 observations, price ≤ 1.05 × the product minimum, and price below its product average. Because the headphones and shoes observations are unchanged, the two expected signals remain unchanged.

9. Troubleshooting reflection

I started by comparing each Check-in 1 column with the data it describes. I realized that the original three-table design had already separated product, store and price facts, so I did not need to split those tables just to make the report look more complicated. To show the normal forms clearly, I reconstructed a flat history sheet, checked the composite business key, and then showed how the original separation removed partial dependencies. I also checked category names: leaving them as free text works for a small example, but a Category table makes filters consistent and removes a transitive dependency from the temporary expanded product view. I kept the same six observation IDs, dates and prices and tested the 3NF SQL in SQLite. The final buy-now results still returned the headphones at Amazon and running shoes at Target. This confirmed that the improved structure did not change the core app behavior.

10. Included submission files

1. This Check-in 2 report (.docx). 2. SQLite 3NF schema, original six sample observations and validation queries (.sql).

Design note: Category is an improvement for consistency and extensibility, not proof that the original three-table Check-in 1 schema was unnormalized. The UNF and 1NF flattened views are instructional reconstructions of the same facts.